

API Documentation - ResidenceCare tRPC

Procedures

Phiên bản: 1.0.0

Ngày tạo: Tháng 5 năm 2026

Tác giả: Manus AI

1. Giới Thiệu tRPC

tRPC là một framework RPC type-safe cho TypeScript. Thay vì REST API truyền thống, tRPC cung cấp:

- **Type Safety:** Tự động type-checking từ server đến client
- **Automatic Validation:** Dữ liệu được validate tự động
- **Zero API Documentation:** Types là documentation
- **Efficient:** Chỉ gửi dữ liệu cần thiết

1.1 Cách Sử Dụng tRPC từ Frontend

```
import { trpc } from "@lib/trpc";

// Query (GET)
const { data, isLoading, error } = trpc.residents.list.useQuery();

// Mutation (POST/PUT/DELETE)
const mutation = trpc.residents.create.useMutation();
await mutation.mutateAsync({ fullName: "Nguyễn Văn A", ... });
```

2. Authentication Router

2.1 `auth.me` (Query)

Lấy thông tin người dùng hiện tại

Request: Không có parameters

Response:

```
{
  id: number;
  openId: string;
  name: string | null;
  email: string | null;
  role: "admin" | "user";
  createdAt: Date;
  updatedAt: Date;
  lastSignedIn: Date;
}
```

Ví dụ:

```
const user = await trpc.auth.me.useQuery();
console.log(user.data?.name); // "Nguyễn Văn A"
```

2.2 `auth.logout` (Mutation)

Đăng xuất người dùng

Request: Không có parameters

Response:

```
{ success: true }
```

Ví dụ:

```
const logout = trpc.auth.logout.useMutation();
await logout.mutateAsync();
```

3. Residents Router

3.1 residents.list (Query)

Lấy danh sách tất cả cư dân

Quyền: Manager, Supervisor

Request: Không có parameters

Response:

```
Array<{
  id: number;
  fullName: string;
  email: string | null;
  phone: string | null;
  dateOfBirth: Date | null;
  gender: "male" | "female" | null;
  schoolId: number | null;
  programId: number | null;
  roomId: number | null;
  status: "active" | "inactive";
  createdAt: Date;
  updatedAt: Date;
}>
```

Ví dụ:

```
const residents = await trpc.residents.list.useQuery();
residents.data?.forEach(r => console.log(r.fullName));
```

3.2 residents.getById (Query)

Lấy thông tin chi tiết một cư dân

Quyền: Manager, Supervisor

Request:

```
{ id: number }
```

Response: Resident object (như 3.1)

Ví dụ:

```
const resident = await trpc.residents.getById.useQuery({ id: 1 });
```

3.3 residents.create (Mutation)

Tạo cư dân mới

Quyền: Manager, Supervisor

Request:

```
{
  fullName: string;           // Bắt buộc
  email?: string;
  phone?: string;
  dateOfBirth?: Date;
  gender?: "male" | "female";
  schoolId?: number;
  programId?: number;
  roomId?: number;
}
```

Response: Resident object mới tạo

Ví dụ:

```
const newResident = await trpc.residents.create.useMutation();
await newResident.mutateAsync({
  fullName: "Trần Thị B",
  email: "b@example.com",
  schoolId: 1,
  roomId: 1
});
```

3.4 residents.update (Mutation)

Cập nhật thông tin cư dân

Quyền: Manager, Supervisor

Request:

```
{
  id: number;
  fullName?: string;
  email?: string;
  phone?: string;
  dateOfBirth?: Date;
  gender?: "male" | "female";
  schoolId?: number;
  programId?: number;
  roomId?: number;
  status?: "active" | "inactive";
}
```

Response: Resident object được cập nhật

Ví dụ:

```
await trpc.residents.update.useMutation().mutateAsync({
  id: 1,
  email: "new@example.com"
});
```

3.5 residents.delete (Mutation)

Xóa (deactivate) cư dân

Quyền: Manager

Request:

```
{ id: number }
```

Response:

```
{ success: true }
```

Ví dụ:

```
await trpc.residents.delete.useMutation().mutateAsync({ id: 1 });
```

4. Rooms Router

4.1 rooms.list (Query)

Lấy danh sách tất cả phòng

Quyền: Manager, Supervisor

Request: Không có parameters

Response:

```
Array<{
  id: number;
  roomNumber: string;
  capacity: number;
  occupancy: number;
  description: string | null;
  createdAt: Date;
  updatedAt: Date;
}>
```

4.2 rooms.create (Mutation)

Tạo phòng mới

Quyền: Manager

Request:

```
{
  roomNumber: string;
  capacity: number;
  description?: string;
}
```

Response: Room object mới tạo

4.3 rooms.assignResident (Mutation)

Gán cư dân vào phòng

Quyền: Manager, Supervisor

Request:

```
{
  roomId: number;
  residentId: number;
}
```

Response:

```
{ success: true; message: string }
```

4.4 rooms.transferResident (Mutation)

Chuyển cư dân sang phòng khác

Quyền: Manager, Supervisor

Request:

```
{
  residentId: number;
  newRoomId: number;
}
```

Response:

```
{ success: true; message: string }
```

5. Attendance Router

5.1 attendance.checkIn (Mutation)

Ghi nhận check-in

Quyền: Manager, Supervisor

Request:

```
{
  residentId: number;
  checkInTime: Date;
  notes?: string;
}
```

Response:

```
{
  id: number;
  residentId: number;
  date: Date;
  checkInTime: Date;
  checkOutTime: Date | null;
  status: "present" | "absent" | "late";
}
```

5.2 attendance.checkOut (Mutation)

Ghi nhận check-out

Quyền: Manager, Supervisor

Request:

```
{
  residentId: number;
  checkOutTime: Date;
  notes?: string;
}
```

Response: Attendance object

5.3 attendance.listSchedules (Query)

Lấy danh sách lịch sinh hoạt

Quyền: Tất cả

Request: Không có parameters

Response:

```
Array<{
  id: number;
  name: string;
  type: "check_in" | "check_out" | "meal" | "study_hour" | "curfew" |
"activity";
  scheduledTime: string; // "HH:mm"
  isDaily: boolean;
  daysOfWeek?: number[];
  createdAt: Date;
}>
```

5.4 attendance.createSchedule (Mutation)

Tạo lịch sinh hoạt mới

Quyền: Manager, Supervisor

Request:

```
{
  name: string;
  type: "check_in" | "check_out" | "meal" | "study_hour" | "curfew" |
"activity";
  scheduledTime: string;
  isDaily?: boolean;
  daysOfWeek?: number[];
}
```

Response: Schedule object

6. Tasks Router

6.1 tasks.list (Query)

Lấy danh sách công việc

Quyền: Manager, Supervisor

Request: Không có parameters

Response:

```
Array<{
  id: number;
  taskTypeId: number;
  assignedTo: number;
  dueDate: Date;
  status: "pending" | "in_progress" | "completed";
  notes: string | null;
  createdAt: Date;
  updatedAt: Date;
}>
```

6.2 tasks.create (Mutation)

Tạo công việc mới

Quyền: Manager, Supervisor

Request:

```
{
  taskTypeId: number;
  assignedTo: number;
  dueDate: Date;
  notes?: string;
}
```

Response: Task object

6.3 `tasks.updateStatus` (Mutation)

Cập nhật trạng thái công việc

Quyền: Manager, Supervisor

Request:

```
{
  taskId: number;
  status: "pending" | "in_progress" | "completed";
}
```

Response: Task object

7. Fees Router

7.1 `fees.listFeeTypes` (Query)

Lấy danh sách loại phí

Quyền: Manager, Accountant

Request: Không có parameters

Response:

```
Array<{
  id: number;
  name: string;
  code: string;
  amount: string; // Decimal
  billingCycle: "monthly" | "quarterly" | "yearly";
  description: string | null;
  isActive: boolean;
  createdAt: Date;
}>
```

7.2 fees.listDebts (Query)

Lấy danh sách công nợ

Quyền: Manager, Accountant

Request:

```
{
  residentId?: number;
  status?: "unpaid" | "paid" | "overdue";
}
```

Response:

```
Array<{
  id: number;
  residentId: number;
  feeTypeId: number;
  amount: string;
  billingMonth: string; // "YYYY-MM"
  dueDate: Date;
  status: "unpaid" | "paid" | "overdue";
  createdAt: Date;
}>
```

7.3 fees.recordPayment (Mutation)

Ghi nhận thanh toán

Quyền: Manager, Accountant

Request:

```
{
  debtId: number;
  amount: string;
  paymentMethod: "cash" | "bank_transfer" | "check";
  notes?: string;
}
```

Response:

```
{
  id: number;
  debtId: number;
  amount: string;
  paymentDate: Date;
  paymentMethod: string;
  notes: string | null;
  createdAt: Date;
}
```

8. Dashboard Router

8.1 dashboard.getStats (Query)

Lấy thống kê tổng quan

Quyền: Manager, Accountant

Request: Không có parameters

Response:

```
{
  totalResidents: number;
  totalRooms: number;
  occupiedRooms: number;
  occupancyRate: number; // 0-100
  todayAttendance: number;
  attendanceRate: number; // 0-100
  pendingTasks: number;
  unpaidDebts: number;
  totalDebtAmount: string;
  recentPayments: number;
}
```

8.2 dashboard.getOccupancyChart (Query)

Lấy dữ liệu biểu đồ sức chứa phòng

Quyền: Manager, Accountant

Request: Không có parameters

Response:

```
Array<{
  roomNumber: string;
  capacity: number;
  occupancy: number;
}>
```

8.3 dashboard.getDebtChart (Query)

Lấy dữ liệu biểu đồ công nợ

Quyền: Manager, Accountant

Request: Không có parameters

Response:

```
Array<{
  status: "unpaid" | "paid" | "overdue";
  count: number;
  amount: string;
}>
```

9. Notifications Router

9.1 notifications.getUnread (Query)

Lấy danh sách thông báo chưa đọc

Quyền: Tất cả

Request: Không có parameters

Response:

```
Array<{
  id: number;
  recipientId: number;
  type: "fee_generated" | "debt_overdue" | "task_assigned" |
"attendance_alert" | "system";
  title: string;
  content: string;
  relatedEntityType: string | null;
  relatedEntityId: number | null;
  isRead: boolean;
  sentAt: Date;
  readAt: Date | null;
}>
```

9.2 notifications.markAsRead (Mutation)

Đánh dấu thông báo đã đọc

Quyền: Tất cả

Request:

```
{ notificationId: number }
```

Response:

```
{ success: true }
```

9.3 notifications.triggerMonthlyDebtGeneration (Mutation)

Kích hoạt sinh công nợ hàng tháng (Admin only)

Quyền: Manager

Request: Không có parameters

Response:

```
{  
  jobName: string;  
  status: "success" | "failure" | "partial";  
  recordsProcessed: number;  
  message: string;  
}
```

10. Error Handling

10.1 Lỗi Phổ Biến

Error Code	Mô Tả	Giải Pháp
UNAUTHORIZED	Chưa đăng nhập	Redirect đến login
FORBIDDEN	Không có quyền truy cập	Kiểm tra role
NOT_FOUND	Dữ liệu không tồn tại	Kiểm tra ID
CONFLICT	Dữ liệu trùng lặp	Kiểm tra unique constraints
INTERNAL_SERVER_ERROR	Lỗi server	Liên hệ support

10.2 Xử Lý Lỗi từ Frontend

```
const mutation = trpc.residents.create.useMutation();

try {
  await mutation.mutateAsync(data);
  toast.success("Tạo cư dân thành công");
} catch (error) {
  if (error.code === "UNAUTHORIZED") {
    // Redirect to login
  } else if (error.code === "FORBIDDEN") {
    toast.error("Bạn không có quyền thực hiện hành động này");
  } else {
    toast.error("Có lỗi xảy ra");
  }
}
```

11. Pagination & Filtering

11.1 Pagination

```
const residents = await trpc.residents.list.useQuery({
  skip: 0,
  take: 20,
});
```

11.2 Filtering

```
const debts = await trpc.fees.listDebts.useQuery({
  residentId: 1,
  status: "unpaid",
});
```

12. Rate Limiting

Không có rate limiting được áp dụng hiện tại. Trong production, nên thêm:

- 100 requests/phút cho mỗi IP
- 1000 requests/giờ cho mỗi user
- Exponential backoff cho retry

13. Versioning

API hiện tại là **v1**. Khi có breaking changes, sẽ tạo v2 mới.

Phiên bản tài liệu: 1.0

Cập nhật lần cuối: Tháng 5 năm 2026

Trạng thái: Production Ready